



Mappeoppgave: Millions

IDATx2003 Programmering 2, våren 2026

Revisjonshistorikk

Versjon	Dato	Endring
1.0	26.01.2026	Mappe del 1
2.0	16.02.2026	Mappe del 2 lagt til
2.1	27.02.2026	Oppdatert diagram for transaksjonsklasser (committed er nå protected)
3.0	16.03.2026	Mappe del 3 lagt til
3.1	20.04.2026	Oppdatert innleveringsdato

Innhold

<i>Innledning</i>	1
Mappens tre deler	1
Før dere begynner	1
<i>Del 1: Grunnleggende klasser og logikk</i>	3
Oppsett av prosjektet	3
Modellen vår	3
Stock og Share	4
Portfolio	5
Transaksjoner	5
Player	8
Exchange	9
Klient som kjører et spill (frivillig)	10
<i>Del 2: Filhåndtering, statistikk og status</i>	11
Filhåndtering	11
Statistikk	12
Nettoverdi	13
Status	13
<i>Del 3: Designmønstre, GUI og egne ideer</i>	14
Designmønstre	14
GUI	15
Videre arbeid	16
<i>Viktige sjekkpunkter</i>	17
<i>Illustrasjoner</i>	19

Figurliste

Figur 1: Klassen Stock.....	4
Figur 2: Klassen Share	4
Figur 3: Portfolio-klassen	5
Figur 4: Kalkulatorklasser	6
Figur 5: Transaksjonsklasser	7
Figur 6: Klassen TransactionArchive	7
Figur 7: Player-klassen	8
Figur 8: Exchange-klassen	9
Figur 9: Nye metoder i Stock-klassen	12
Figur 10: Statistikk i Exchange-klassen	12
Figur 11: MVC og Observer	14

Innledning

I mappeoppgaven skal dere utvikle et aksjespill. Når spillet er ferdig skal man kunne:

- starte et nytt spill med en gitt startkapital, med aksjedata lest fra fil
- hente ut aksjeinformasjon og statistikk
- kjøpe og selge andeler
- se alle transaksjoner som er gjennomført (kjøp og salg)
- få informasjon om vinnere og tapere på markedet
- avansere til ny uke, som medfører endring i priser
- hente ut sin nettoverdi og status
- selge unna hele sin beholdning og avslutte programmet

En mer detaljert liste over funksjonelle krav presenteres i del 3.

Mappens tre deler

Mappen består av et prosjekt i tre deler (teller 70%) og en rapport (teller 30%). Delene presenteres etappevis gjennom semesteret:

- I del 1 er fokuset på grunnleggende klasser og logikk
- I del 2 skal koden utvides med blant annet filhåndtering og statistikk
- I del 3 skal dere implementere et grafisk brukergrensesnitt, anvende designmønster, samt utvide løsningen basert på egne ideer

Hver del bygger på foregående del. Når ny del publiseres vil dere få muntlig tilbakemelding på arbeidet som er gjort så langt, med mulighet til å justere og forbedre løsningen. Dere vil få i alt tre slike tilbakemeldinger. Den fullstendige mappen leveres i Inspira til slutt.

Før dere begynner

Følgende krav og betingelser gjelder for alle oppgavene:

Enhetstesting

Dere må lage enhetstester (både positive og negative) for den delen av koden som er forretningskritisk, altså for den koden som er viktigst for å oppfylle sentrale krav. Feil her vil kunne få store negative konsekvenser for programmet.

Unntakshåndtering

Uønskede hendelser og tilstander som forstyrrer normal flyt skal håndteres på en god måte. Håndteringen skal være rimelig og balansert (ikke for mye, ikke for lite) med det formål å gjøre koden mer robust. Vi gjør oppmerksom på at unntakshåndtering ikke er eksplisitt angitt i oppgavene under. Dere må selv avgjøre når og hvordan unntak skal implementeres.

Kodekvalitet

Det forventes høy kodekvalitet i prosjektet. Bruk verktøy som CheckStyle (med Google sine regler) og SonarLint. Begge verktøy finnes som plugins til IntelliJ og VS Code.

Versjonskontroll

Prosjektet skal legges under versjonskontroll. Krav til versjonskontroll er ytterligere beskrevet i del 1 – Oppsett av prosjektet.

Prosjektrapport

Det skal skrives en rapport for prosjektet. I rapporten skal dere forklare hvordan løsningen er bygget opp, hvilke valg som er tatt underveis, hvordan dere har anvendt designprinsipper, reflektere rundt bruk av KI-verktøy osv. Rapporten skal være på 2500-3000 ord. I tillegg kommer figurer og eventuelt små kodesnutter for å vise hvordan utvalgte problemer er løst. Det er lurt å begynne på rapporten tidlig. Dokumentmal er tilgjengelig i Blackboard.

Bruk av KI-verktøy

Det er tillatt å anvende KI-verktøy som støtte for å løse mappen, så lenge

- verktøyene først og fremst benyttes som sparringspartner/rådgiver og oppslagsverk
- eventuell kode som foreslås gjennomgås kritisk/bearbeides før bruk
- dere kommenterer i koden der dere har fått hjelp av slike verktøy
- bruken dokumenteres i rapporten (hvilke verktøy ble benyttet til hva, gjerne med eksempler på prompts og svar)

Del 1: Grunnleggende klasser og logikk

I denne første delen skal dere opprette et nytt prosjekt og fokusere på grunnleggende klasser og logikk. Når dere har jobbet dere gjennom oppgavene skal resultatet vises til en lærings-assistent. Det er ikke et krav at alle oppgavene skal være fullstendig løst før fristen, men de må som minimum være påbegynt for å få godkjent. Solid og jevn innsats vil gi et godt grunnlag for konstruktive tilbakemeldinger. Vi gjør oppmerksom på at dere også skal levere det som er gjort i Blackboard.

Det kan være lurt å lese gjennom hele dokumentet før dere begynner på oppgavene.

Oppsett av prosjektet

Opprett et tomt Maven-prosjekt og gi prosjektet en fornuftig groupId og artifactId. Prosjektet må følge kravene til konfigurasjon og versjoner som er angitt i BB. Når dere svarer på kodeoppgavene under skal filer lagres iht standard Maven-oppsett: enhetstester legges i katalogen "test/java", eventuelle ressursfiler (bilder, konfigurasjon osv) legges i "main/resources", mens resten av koden hører hjemme i katalogen "main/java". Det skal være mulig å bygge, teste og pakke med Maven fra kommandolinja. Det betyr bl.a. at kommandoen «mvn clean package» skal kjøre uten feil.

Prosjektet skal være underlagt versjonskontroll og være koblet mot et sentralt repository:

- legg først koden under lokal versjonskontroll
- opprett så et nytt sentralt repo (tomt prosjekt) med samme navn på GitLab/GitHub (studenter på campus Ålesund og campus Gjøvik skal bruke GitHub Classroom)
- til slutt kobler dere lokalt repository mot sentralt repository

Dere kan nå samarbeide og pushe mot en felles kodebase. Sjekk inn (commit) jevnlig. Husk at hver commit-melding skal beskrive endringene som er gjort på en kort og konsis måte. Hvis dere sjekker inn på slutten av en oppgave, men senere trenger å gjøre endringer i koden og sjekke inn på nytt, så er det selvfølgelig helt greit. Det er også viktig å merke seg at .git-katalogen skal være med når dere leverer besvarelsen i BB (.git-katalogen er en "usynlig" mappe i rot-katalogen til prosjektet deres som inneholder all versjonshistorikk).

Modellen vår

Før vi begynner å kode kan det være greit med en kort beskrivelse av de grunnleggende klassene i spillet.

En spiller (Player) kan kjøpe og selge andeler (Share) i en aksje (Stock). Kjøp (Purchase) og salg (Sale) foregår på en børs (Exchange), og representerer finansielle transaksjoner (Transaction). En transaksjon har verdier og kostnader som beregnes av en dedikert kalkulator (TransactionCalculator). Vi opererer med ulike kalkulatorer for kjøp (PurchaseCalculator) og salg (SaleCalculator). Andeler oppbevares i en portefølje (Portfolio), og transaksjonshistorikk lagres i et arkiv (TransactionArchive).

Det er lov å gjøre endringer i modellen dersom det fører til en mer robust arkitektur med løsere koblinger og høyere kodekvalitet. Eventuelle endringer må dokumenteres og begrunnes i prosjektrapporten.

Stock og Share

I dagligtale brukes gjerne begrepene Stock og Share om hverandre. Rent teknisk er det imidlertid vanlig å skille mellom andelen man eier og selve aksjen, og det er denne forståelsen vi legger til grunn for modellen vår.

Stock

Klassen representerer en aksje i et selskap. Den har et unikt symbol, f.eks. «AAPL» for Apple Inc. Aksjer selges på børse (Exchange) for en salgspris som oppdateres ukentlig.

Stock
- symbol: String - company: String - prices: List<BigDecimal>
+ Stock(symbol: String, company: String, salesPrice: BigDecimal) + getSymbol(): String + getCompany(): String + getSalesPrice(): BigDecimal + addNewSalesPrice(price: BigDecimal)

Figur 1: Klassen Stock

Metoden getSalesPrice() skal returnere gjeldende salgspris, som er den siste prisen som ble lagt til i prices.

Share

Et kjøp resulterer i en andel ved klassen Share. Andelen har informasjon om hva slags aksje som ble kjøpt, hvor mye som ble kjøpt (kvantitet), og kjøpspris.

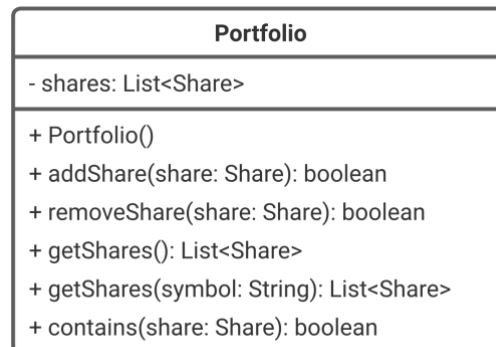
Share
- stock: Stock - quantity: BigDecimal - purchasePrice: BigDecimal
+ Share(stock: Stock, quantity: BigDecimal, purchasePrice: BigDecimal) + getStock(): Stock + getQuantity(): BigDecimal + getPurchasePrice(): BigDecimal

Figur 2: Klassen Share

Legg merke til at vi bruker BigDecimal og ikke float for å representerer desimaltall. Datatypen float har utfordringer knyttet til nøyaktighet¹. BigDecimal er en del av Java APIet og den anbefalte løsningen når vi må regne med penger.

Portfolio

Vi lagrer andelene til en spiller i en portefølje. Portfolio-klassen er relativt enkel; den har en liste for andeler og metoder for å legge til, fjerne, hente ut, samt sjekke om en andel finnes i porteføljen:



Figur 3: Portfolio-klassen

Transaksjoner

Når vi kjøper og selger aksjer gjennomfører vi en finansiell transaksjon. En transaksjon er knyttet til en andel og en gitt uke. Det er verdier og kostnader knyttet til transaksjonene:

- Bruttoverdi: verdi før avgifter
- Kurtasje (commission): en avgift som betales til megleren
- Skatt (tax): en avgift som betales til staten
- Totalverdi: verdi etter avgifter

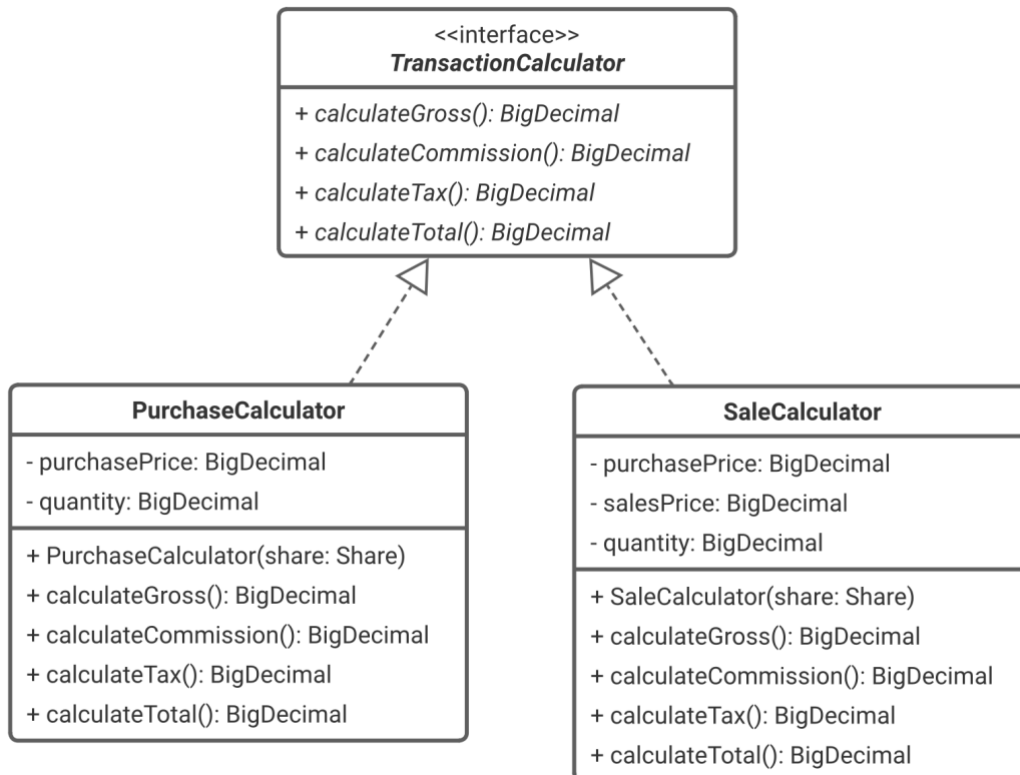
Vi velger å skille ut koden som utfører beregninger av verdier og kostnader i egne klasser.

TransactionCalculator, PurchaseCalculator og SaleCalculator

Alle transaksjoner har egenskapene som beskrevet over, men beregningene for kjøp og salg gjøres forskjellig. Vi definerer derfor de nødvendige beregningsmetodene i et felles grensesnitt (TransactionCalculator), og implementerer de i henholdsvis PurchaseCalculator og SaleCalculator.

Begge klassene har en konstruktør som tar inn en andel. PurchaseCalculator baserer sine beregninger på andelens kjøpspris og kvantitet, mens SaleCalculator i tillegg henter ut salgspris.

¹ Se <https://stackoverflow.com/questions/3730019/why-not-use-double-or-float-to-represent-currency>



Figur 4: Kalkulatorklasser

For PurchaseCalculator gjelder følgende:

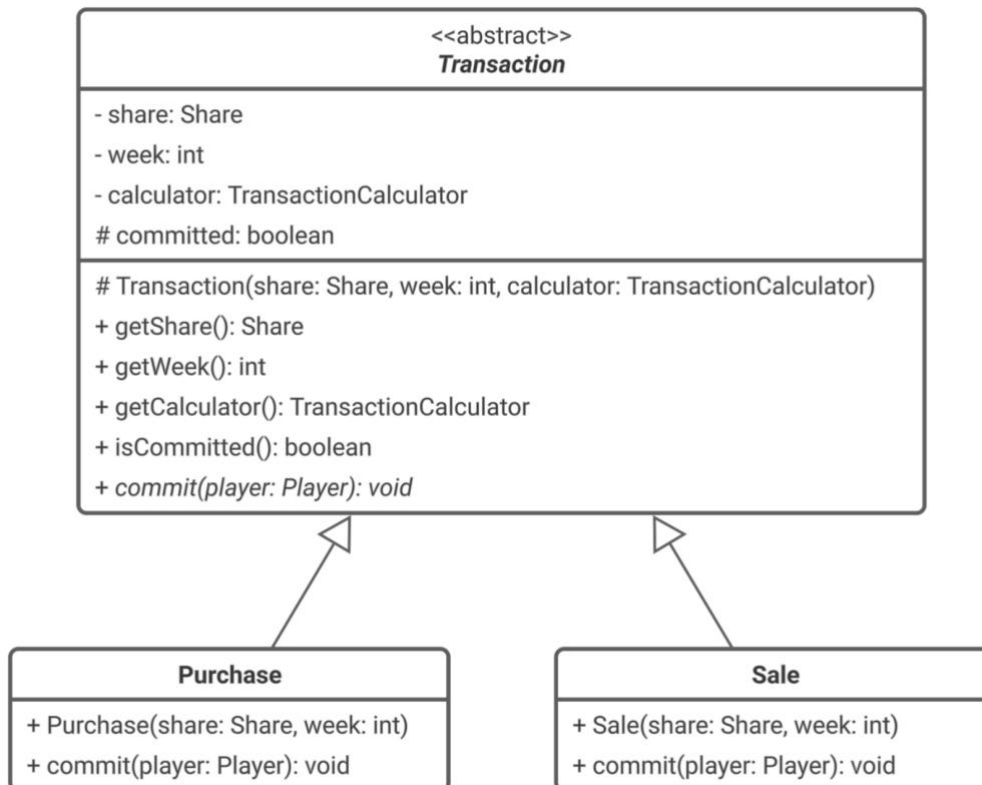
- `calculateGross()`: bruttoverdi, altså kjøpspris $\times$ kvantitet
- `calculateCommission()`: en avgift på 0.5% av bruttoverdi
- `calculateTax()`: ingen skatt ved kjøp
- `calculateTotal()`: total kostnad, dvs bruttoverdi + avgift + skatt

For SaleCalculator blir det litt annerledes:

- `calculateGross()`: bruttoverdi, altså salgspris $\times$ kvantitet
- `calculateCommission()`: en avgift på 1% av bruttoverdi
- `calculateTax()`: 30% skatt på gevinst
 - Gevinsten kan beregnes som bruttoverdi – avgift – kjøpskostnader
 - Vi kan forenkle litt og si at kjøpskostnader er kjøpspris $\times$ kvantitet
- `calculateTotal()`: total salgsverdi, altså bruttoverdi – avgift – skatt

Transaction, Purchase og Sale

Kjøp og salg har det til felles at de er knyttet til en andel og en uke, og at de bruker en kalkulator for beregninger. Men det er også vesentlige forskjeller. De bruker logisk nok ulike kalkulatorer, og selve gjennomføringen er forskjellig. Da er det naturlig å samle det som er felles i en abstrakt superklasse som vi kaller Transaction. Vi delegerer gjennomføringen til en abstrakt `commit`-metode, som implementeres i subclassene Purchase og Sale:



Figur 5: Transaksjonsklasser

Commit-metodene avhenger av Player-klassen, så vi må vente litt med å implementere disse. Enn så lenge kan vi opplyse om at alle transaksjoner skal betraktes som unike; den samme transaksjonen kan ikke gjennomføres flere ganger. Derfor har vi et flagg i super-klassen kalt `committed`, som settes til `true` første gangen `commit` kalles.

TransactionArchive

Alle fullførte transaksjoner skal oppbevares i et arkiv. Klassen **TransactionArchive** har følgende egenskaper:

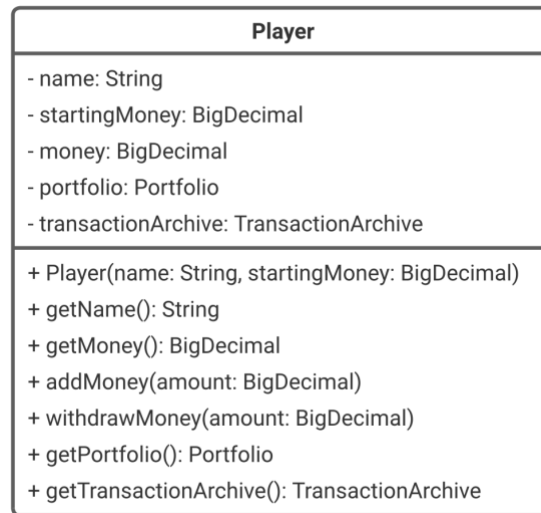


Figur 6: Klassen TransactionArchive

Den siste metoden – `countDistinctWeeks()` – skal telle antall uker med faktisk handel, og blir nyttig i del 2 av mappen.

Player

Player-klassen ser slik ut:



Figur 7: Player-klassen

En ny spiller har et navn og startkapital. Merk at vi opererer med to attributter for penger; `startingMoney` og `money`. Beløpene vil være like ved oppstart, men det er `money` som skal representere den til enhver tid gjeldende pengebeholdningen. Vi trenger imidlertid å vite hva vi startet med når vi senere skal beregne status i spillet, derfor lagrer vi også startkapitalen. Konstruktøren må også instansiere en ny portefølje og et tomt transaksjonsarkiv.

Nå som `Player`-klassen er implementert, kan vi gyve løs på `commit`-metodene i `Purchase` og `Sale`.

`Purchase` sin `commit`-metode fullfører et kjøp ved å:

1. trekke totalkostnadene fra spillerens pengebeholdning
2. legge til andelen i spillerens portefølje, og
3. lagre transaksjonen i spillerens transaksjonsarkiv

Flagget `committed` settes til `true` hvis alt går bra. Det skal ikke være mulig å fullføre et kjøp hvis spilleren ikke har tilstrekkelig med penger, eller hvis transaksjonen er gjennomført tidligere.

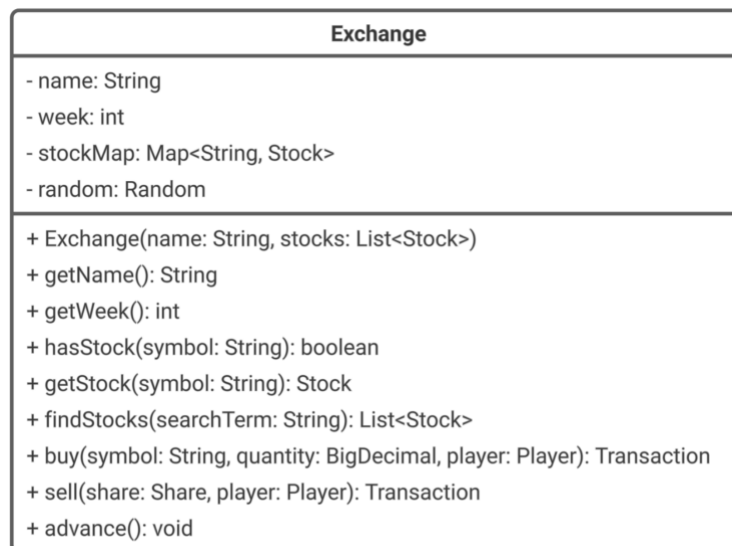
`Sale` sin `commit`-metode fullfører et salg ved å:

1. legge til total-verdien til spillerens pengebeholdning
2. fjerne andelen fra spillerens portefølje, og
3. lagre transaksjonen i spillerens transaksjonsarkiv

Flagget committed settes til true hvis alt går bra. Det skal ikke være mulig å fullføre et salg hvis spilleren ikke eier andelen som skal selges, eller hvis transaksjonen er gjennomført tidligere.

Exchange

Exchange er den siste klassen i modellen vår, og representerer ganske enkelt en børs hvor man kan handle aksjer:



Figur 8: Exchange-klassen

Klassen har en konstruktør som tar inn et navn og en liste med aksjene som kan handles på børsen. Internt lagres aksjene i et Map, hvor «symbol» brukes som nøkkel. Ukenummer settes til 1.

Det skal være mulig å kjøpe og selge aksjer som er notert på børsen via buy- og sell-metodene. En handel fullføres som en transaksjon.

Metoden `advance()` brukes for å gå videre til neste handelsuke, ved å inkrementere ukenummeret og oppdatere prisene. Oppdateringen skjer ved å gå gjennom hver eneste aksje og enten øke eller senke prisen. Ny pris må være tilfeldig for hver aksje, og endringen bør ikke være for stor. Tilfeldighet kan oppnås ved å bruke Random-klassen² i Java APllet.

Det skal være mulig å hente ut informasjon om aksjer:

- `getStock` returnerer ganske enkelt aksjen for et gitt symbol
- `findStocks` returnerer en liste med alle aksjer som inneholder et søkeord. Vi må sjekke både symbolet og selskapsnavnet. Et søk på «Go» vil f.eks. kunne returnere både «GOOGL – Alphabet Inc» og «WFC – Wells Fargo».

Klassen har også metoder for å hente ut navn og ukenummer.

² <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/Random.html>

Klient som kjører et spill (frivillig)

I utgangspunktet skal enhetstestene være nok for å verifisere at modellen fungerer som forventet. Men hvis dere ønsker kan dere også lage en enkel klient som gjør det mulig å kjøre et spill fra kommandolinja.

Del 2: Filhåndtering, statistikk og status

I denne delen skal dere utvide programmet med filhåndtering og statistikk. Dere skal også legge til metoder som gjør det mulig å beregne spillerens nettoverdi og status. Til slutt må dere levere besvarelsen og gjennomføre en godkjenningssamtale, akkurat som for del 1.

Filhåndtering

Programmet skal ha støtte for filhåndtering. Når vi oppretter et nytt spill så vil det være nyttig å kunne lese aksjedata fra en vilkårlig fil. En fil kan f.eks. representere aksjene på Oslo børs, mens en annen kan inneholde aksjer notert på Nasdaq. Eksempel-fil med aksjedata er tilgjengelig på Blackboard.

Det skal også være mulig å lagre aksjedata til fil. Dette er ikke nødvendigvis funksjonalitet som vil være tilgjengelig for brukeren i GUI, men vi ønsker at APIet vårt skal ha slik støtte med tanke på eventuelle fremtidige utvidelser.

Format

Aksjedata lagres i et bestemt format og som en CSV-fil (tekstfil med komma-separerte verdier). En linje i fila kan enten være en kommentar, blank, eller info om en aksje:

- kommentar: starter med # (hashtag), og skal ignoreres (hoppes over)
- blank: tom linje som skal ignoreres
- informasjon om en aksje (Stock), på formen symbol,name,price

Prisen som oppgis er startprisen, altså den prisen som gjelder for første handelsuke. Det kan også være verdt å merke seg at punktum brukes som desimaltegn.

Eksempel på innhold og format:

```
# Top 500 US Stocks by Market Cap
# Ticker,Name,Price

NVDA,Nvidia,191.27
AAPL,Apple Inc.,276.43
MSFT,Microsoft,404.68
...
```

Statistikk

Nye metoder i Stock-klassen

Når vi handler aksjer kan det være interessant å vite om prishistorikk og endringer i pris. For å kunne se statistikk om enkeltaksjer utvider vi Stock-klassen med metodene som vises i følgende klassesdiagram:

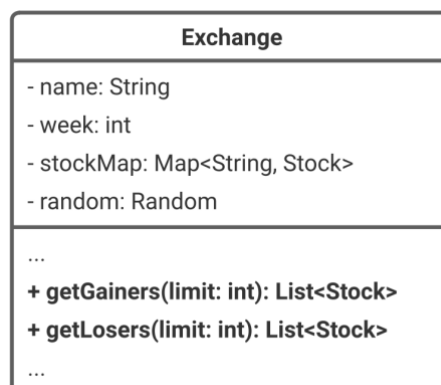


Figur 9: Nye metoder i Stock-klassen

Den første metoden returnerer en liste med historiske priser. Vi gjør det enkelt og sier at det er alle prisene som noensinne har vært registrert. De to neste metodene returnerer henholdsvis høyeste og laveste registrerte pris. Den siste metoden skal returnere differansen mellom siste og nest siste registrerte pris. Har vi kun én registrert pris skal dette tolkes som ingen endring.

Statistikk i Exchange-klassen

Det kan også være nyttig å vite om vinnere og tapere på børsen. Vi utvider derfor Exchange-klassen med to metoder. Metoden `getGainers` skal liste opp aksjene som har hatt størst positiv endring i pris siden sist uke, mens `getLosers` skal liste opp aksjene som har gjort det dårligst på børsen siden sist uke. Begge metodene tar inn `limit` som parameter, for å begrense antall aksjer som returneres.



Figur 10: Statistikk i Exchange-klassen

Nettoverdi

En spiller vet hele tiden hvor mye penger hen har å rutte med (money). Men det hadde også vært kjekt å vite hva man sitter på av verdier totalt, altså verdien av porteføljen pluss pengebeløpet. For å få tilgang til spillerens nettoverdi må vi gjøres følgende:

1. Utvid Portfolio-klassen med metoden `getNetWorth()`. Metoden skal returnere total salgsverdi for alle andelene i porteføljen. `SaleCalculator` er nyttig her.
2. `Player`-klassen skal ha sin egen `getNetWorth()`. Metoden skal returnere spillerens gjenværende pengebeholdning pluss verdien av porteføljen.

Status

Vi kan si noe om fremgangen til en spiller ved å sammenlikne nåværende nettoverdi og startkapital. Hvis differansen er positiv har spilleren tjent penger. Hvor lenge man har spilt og hvor mye penger man har tjent avgjør hva slags nivå man havner på:

- **Novice:** startnivå, ingen krav til gevinst eller handel
- **Investor:** nivå for spillere som har handlet i minst 10 uker og økt nettoverdier sin med minst 20%
- **Speculator:** nivå for spillere som har handlet i minst 20 uker og som minimum doblet nettoverdier sin

Det er opp til dere å omsette betingelsene til kode. Det er imidlertid et krav at `Player`-klassen har en metode som returnerer spillerens status. Denne skal senere kunne brukes i GUI for å vise fremgang i sanntid.

Del 3: Designmønster, GUI og egne ideer

I denne siste delen skal dere anvende designmønster, samt skissere og kode et grafisk brukergrensesnitt. I tillegg skal dere utvide spillet basert på egne ideer. Ta utgangspunkt i det dere allerede har gjort i de to andre delene når dere løser oppgavene. Som vanlig skal dere levere besvarelsen i Blackboard og gjennomføre en godkjenningssamtale.

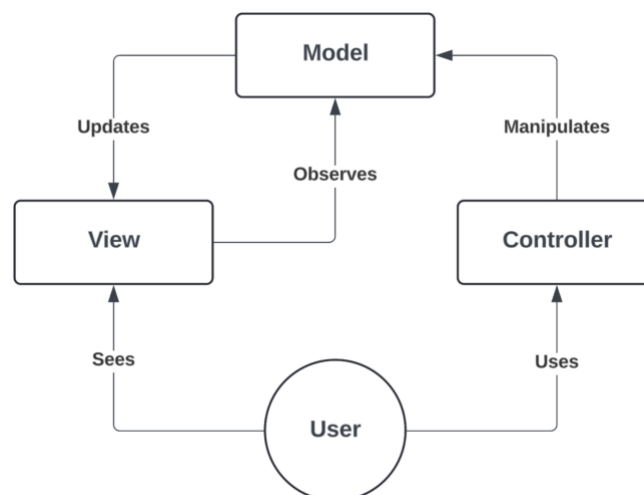
Det er mulig å jobbe med prosjektet helt frem til endelig innlevering i Inspira. Inspira vil være åpen for innlevering i tidsrommet 26.05 09:00 - 27.05 14:00.

Designmønster

Factory

Implementer en fabrikk som gjør det enkelt å opprette ulike typer transaksjoner. Bruk fabrikkene der det er hensiktsmessig.

MVC og Observer



Figur 11: MVC og Observer

Det grafiske brukergrensesnittet skal designes i henhold til Model-View-Controller³. Når en bruker utfører en handling vil det gjerne føre til en endring i modellen. Det er kontrollerens oppgave å utføre denne endringen. Samtidig er det viktig at brukergrensesnittet oppdateres når tilstanden i modellen endres. Dette er et klassisk eksempel hvor Observer passer godt. Modellen skal være observerbar slik at den kan gi beskjed til viewet (observatøren) om endringer i tilstand. Viewet er på sin side ansvarlig for å vise frem endringene til brukeren.

³ MVC omtales ofte som et designmønster, selv om det kanskje er mer presist å kalle det for et arkitektonisk mønster. Se <https://stackoverflow.com/questions/1866821/is-mvc-a-design-pattern-or-architectural-pattern> for mer info.

Decorator, builder...

Tenk gjennom: er det andre designmønstre som bør implementeres? Fornuftig bruk av designmønstre vil telle positivt i en helhetsvurdering. Her bør vi presisere at det ikke er noe poeng å implementere et designmønster hvis det ikke gir positive ringvirkninger ellers i koden. Builder er f.eks. ikke særlig relevant hvis klassen har få attributter, og observer har liten nytte hvis det ikke finnes klasser som faktisk observerer endringene.

GUI

Spillet skal ha et grafisk brukergrensesnitt. I innledninga ga vi en kort oversikt over de funksjonelle kravene. Under følger en mer detaljert liste over krav og betingelser i spillet:

- Det skal være mulig å starte et nytt spill. Ved oppstart skal brukeren kunne angi navn, startkapital, og en vilkårlig fil med aksjedata:
 - Brukeren må selv kunne velge fil fra hvor som helst på datamaskinen
 - Programmet må støtte formatet som er angitt i del 2
- Det må være mulig å hente ut informasjon om aksjene på børsen:
 - Systemet må tilby søk, filtrering e.l. som gjøre det lett å finne enkeltaksjer
 - Det skal være mulig å hente ut statistikk om hver aksje (høyeste og laveste pris, prishistorikk osv)
 - Programmet skal kunne vise info om ukens vinnere og tapere
- Det skal være mulig å kjøpe og selge andeler:
 - Spillerens portefølje må være lett synlig, slik at man får et overblikk over hvilke andeler man eier og kan selge
 - Før man fullfører en transaksjon bør man få informasjon om verdier og kostnader knyttet til transaksjonen
 - Etter at man har fullført en transaksjon må systemet vise en kvittering (transaksjonsrapport)
- Gjennomførte transaksjoner må være tilgjengelige:
 - Det bør være enkelt å søke/filtrere i en transaksjonssoversikt
 - Det må være mulig å få detaljert info om hver enkelt transaksjon (kvantitet, pris, kostnader osv)
- Når man er ferdig med ukas handel må man kunne avansere til ny uke:
 - Dette medfører endring i priser, som naturlig nok må gjenspeiles ellers i programmet
- GUI må opplyse om spillerens nettoverdi og status i sanntid
- Det skal være mulig å selge unna hele beholdningen og avslutte programmet

Det er viktig at applikasjonen er brukervennlig. Det bør være mulig å forstå hva man skal gjøre uten en eksplisitt bruksanvisning, layouten bør være responsiv og oversiktlig, og feilsituasjoner må håndteres på en fornuftig måte.

Skisser

Før dere begynner å kode GULET er det lurt å lage noen skisser som illustrerer utseende og layout. Dere kan bruke et dedikert verktøy som Balsamiq Wireframes eller Figma, eller ganske enkelt tegne med blyant og papir. Skissene trenger ikke være så detaljrike, poenget er få innsikt i den generelle strukturen og hva slags komponenter man trenger. Samtidig er det viktig at skissene er oversiktlige og lesbare. Gode skisser gjør det lettere å kode brukergrensesnittet, og i tillegg se om det er komponenter som kan gjenbrukes.

Implementasjon

GUI-applikasjonen skal implementeres med JavaFX. View og komponenter skal kodes i Java, som betyr at det ikke er lov å bruke FXML. Selv om vi krever at dere ikke implementerer GUI ved bruk av FXML, så er det selvsagt ingenting i veien for å bruke verktøyet Scenebuilder for å eksperimentere, se muligheter og lage skisser.

Som tidligere nevnt skal MVC og Observer anvendes i løsningen.

Videre arbeid

Frem til nå har dere jobbet ut ifra veldefinerte oppgaver og krav. Underveis har dere kanskje fått egne ideer. I denne oppgaven oppfordrer vi dere til å være kreative og gå utenfor de begrensningene vi har satt. Det er dere som bestemmer, men her er noen forslag:

- Mulighet til å ta opp lån, som må betales tilbake med renter
- Tilfeldige hendelser som påvirker aksjeprisen
- Ulike vanskelighetsgrader
- Grafisk visualisering av prisutviklingen (JavaFX har støtte for å tegne diagrammer)
- Kunne følge med på utvalgte aksjer eller få varsel om aksjer med stor endring i pris
- Profilerings/analyse med anbefaling av salg/kjøp
- Multimedia for økt innlevelse og engasjement (lyd, bilde, animasjon, film)
- Støtte for flere språk (internasjonalisering)
- Muligheten til å lagre et spill og fortsette senere (tips: JSON⁴)
- Highscoreliste
- Tak på antall aksjer som kan handles per uke
- Kunne investere basert på beløp i tillegg til kvantitet
- Funksjon for å legge inn og/eller redigere aksjedata før man starter et spill

Det forventes at dere tar prosjektet videre med egne ideer, men poenget er ikke å legge til mest mulig funksjonalitet. Kvalitet er viktigere enn kvantitet. Velg gjerne utvidelser som viser at dere behersker ulike læringsmål i emnet.

⁴ <https://www.json.org/json-en.html>

Viktige sjekkpunkter

Når dere løser mappeprosjektet bør dere dobbeltsjekke følgende:

- Maven:
 - Er prosjektet et Maven-prosjekt med fornuftige prosjekt-verdier og gyldig katalogstruktur?
 - Følges kravene til prosjektkonfigurasjon?
 - Kan man kjøre Maven-kommandoer for å bygge, teste, pakke og starte GUI uten at det feiler?
 - Er det mulig å kjøre disse kommandoene fra terminalen med mvn?
- Versjonskontroll med git:
 - Er prosjektet underlagt versjonskontroll med sentralt repo?
 - Har prosjektet en .gitignore-fil som filtrerer bort irrelevante kataloger og filer?
 - Sjekkes det inn jevnlig?
 - Beskriver commit-meldingene endringene på en kort og konsis måte?
 - Benyttes grener («branches») på en hensiktsmessig måte?
 - Benyttes tags for å merke versjoner?
- Enhetstester:
 - Har enhetstestene beskrivende navn som dokumenterer hva testene gjør?
 - Følger de mønsteret Arrange-Act-Assert?
 - Tas det hensyn til både positive og negative tilfeller?
 - Er testdekningen fornuftig?
 - Benyttes livssyklus-metoder der det er fornuftig (@BeforeEach osv)?
- Er klasser og grensesnitt implementert iht oppgavebeskrivelsen?
 - Dokumenteres og begrunnes eventuelle avvik i rapporten?
- Filhåndtering:
 - Er det mulig å lese aksjedata fra fil på oppgitt format?
 - Har APIet støtte for å skrive aksjedata til fil på oppgitt format?
 - Hensyntas feil i data og format?
 - Lukkes filressurser på en trygg måte etter bruk?
 - Er filhåndteringen implementert på en måte som gjør det enkelt å innføre støtte for andre formater senere?
- Er fabrikken korrekt implementert, og anvendes den ellers i koden?
- Er modellen observerbar (Observer pattern)?
- GUI:
 - Oppfyller GUI kravene og betingelsene som utdypet i del 3?
 - Er GUI designet iht MVC?
 - Er Observer implementert på en fornuftig måte?
 - Er det mulig å forstå hvordan applikasjonen skal brukes uten en eksplisitt bruksanvisning?
 - Er layouten responsiv og oversiktlig?
 - Håndteres feilsituasjoner godt, med beskrivende feilmeldinger osv?
- Kodekvalitet:
 - Er koden godt dokumentert iht JavaDoc-standard?
 - Er koden robust (unntakshåndtering, validering mm)?
 - Har variabler, metoder og klasser beskrivende navn?

- Er klassene gruppert i en logisk pakkestruktur?
 - Har koden god struktur, med løse koblinger og høy kohesjon?
 - Benyttes arv og grensesnitt slik at løsningen er fleksibel og lett å utvide/endre?
 - Benyttes funksjonell programmering (lambda og streams) der det er naturlig?
 - Benyttes andre designmønstre (utover de som er spesifisert i oppgaven) der det er hensiktsmessig?
 - Benyttes linter (f.eks. SonarLint) og tilsvarende verktøy (f.eks. CheckStyle) for kvalitetssjekk?
- Rapport:
 - Har rapporten en fornuftig struktur og et godt språk?
 - Er de formelle kravene oppfylt (innholdsfortegnelse, figur- og tabelliste, sidetall, korrekt referansebruk og referanseliste, vedlegg, maks lengde osv)?
 - Omtaler rapporten temaene/kapitlene som er listet opp i malen, med solide beskrivelser, begrunnelser og refleksjoner?

Illustrasjoner

Alle illustrasjoner og figurer er tilrettelagt av fagstaben i IDATx2003, med unntak av bildet på forsiden, som er utarbeidet av pch.vector/Freepik.